Singapore HDB Resale Price Prediction

This notebook is **Part 2** of the Singapore HDB resale project.
Part 1 focused on **visualization, mapping, and exploratory analysis**.
This notebook shifts to **predictive modeling**: can historical resale transactions be used to estimate what a flat would sell for?

Main goal

Build and compare multiple supervised machine-learning models that predict `resale_price` from:

- flat size and configuration
- age / lease-related variables
- storey information
- sale timing
- geospatial location
- neighborhood / street context

Practical motivation

The most practical question behind this notebook is simple:

> **If I wanted to sell my own HDB flat today, can I use past market transactions to estimate a reasonable market price?**

That makes this notebook more than just a machine-learning exercise. It is also a valuation-oriented workflow:

1. prepare a clean transaction dataset
2. engineer economically meaningful features
3. compare model families fairly
4. interpret what the strongest model has learned
5. apply the model to a real flat as a pricing aid

Learning roadmap

The notebook is organized as an end-to-end modeling workflow:

1. import and inspect the prepared transaction data
2. engineer predictive features from raw columns
3. split the data into train / validation / test sets
4. define reusable preprocessing and evaluation helpers
5. train several model families
6. compare out-of-sample performance
7. visualize actual vs predicted values
8. interpret the best tree model with feature importance and SHAP
9. use dimension reduction for intuition-building
10. summarize what the results imply for practical pricing

Why compare multiple models?

Different algorithms capture different kinds of patterns:

- **linear models** provide an interpretable baseline
- **polynomial regression** allows limited curvature while staying relatively simple
- **tree ensembles** capture non-linear interactions that are common in housing data
- **k-NN** acts like an automated “similar recent sales” benchmark
- **neural networks** offer another flexible non-linear function class

The point is not just to fit one fancy model.
It is to learn **which modeling assumptions work best for HDB resale prices** and whether the performance is strong enough to be useful in practice.

In [1]:
```
# Optional installs (uncomment only if needed in your environment)
# %pip install tqdm umap-learn shap xgboost lightgbm
```

In [2]:
```
import warnings
warnings.filterwarnings("ignore")

from pathlib import Path
import time
import random

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tqdm.auto import tqdm
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
import shap
import umap

from sklearn.model_selection import train_test_split, ParameterSampler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import seaborn as sns

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)

plt.rcParams["axes.grid"] = True
```

1. Import data

This section loads the cleaned resale transaction file together with the geocoded location information created in the visualization notebook.

Why this notebook depends on Part 1

A major theme of the project is that **location is one of the strongest drivers of housing price**.
Without geocodes, the model would only know broad administrative labels such as town or street name.
With latitude and longitude, it can learn much richer spatial price patterns.

That is why the recommended run order is:

1. `singapore_hdb_resale_visualization.ipynb`
2. `singapore_hdb_resale_prediction.ipynb`

What to check in this section

Before any modeling starts, it is important to verify that:

- the data loaded correctly

- the row count looks reasonable
- numeric columns have plausible ranges
- categorical columns contain the expected values
- resale price has a distribution that matches intuition

This first inspection step helps catch data-quality problems early, before they silently affect model performance later.

Import CSV data

The raw transaction CSV is the main modeling table.

The first few cells deliberately do only **foundational cleaning**:

- convert important fields into numeric form
- standardize the key transaction columns
- remove clearly unusable rows
- keep the table stable before adding more complex features

This is good workflow design.
It separates **basic data integrity work** from **feature engineering**, so a future reader can understand which columns came directly from the source and which were derived later.

In [3]:
```
# Update this path if your CSV is saved somewhere else.
DATA_PATH = Path("outputs/transactions_clean.csv")
df = pd.read_csv(DATA_PATH)
```

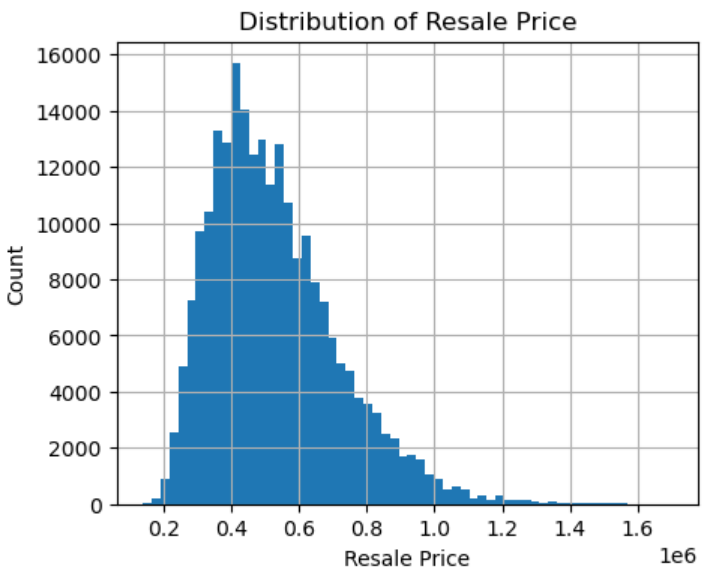
In [4]:
```
# Numeric summary
display(df.describe(include=[np.number]).T)

# Categorical summary
categorical_cols_raw = df.select_dtypes(include="object").columns.tolist()
display(df[categorical_cols_raw].describe().T)
```

	count	mean	std	min	25%	50%	75%	r
floor_area_sqm	226903.0	96.739867	24.017004	31.00000	81.00000	93.00000	112.00000	3.667000e
lease_commence_date	226903.0	1996.480170	14.320886	1966.00000	1985.00000	1997.00000	2012.00000	2.021000e
resale_price	226903.0	527277.202048	188003.504780	140000.00000	388000.00000	495888.00000	632888.00000	1.700000e
psm	226903.0	5510.127306	1629.581749	2090.00000	4353.00000	5230.00000	6259.00000	1.614900e
postal	226903.0	563399.845414	190571.425112	50004.00000	510250.00000	560710.00000	730362.00000	8.251950e
latitude	226903.0	1.368226	0.042984	1.27038	1.33738	1.36754	1.39681	1.457070e
longitude	226903.0	103.841205	0.070973	103.68523	103.78046	103.84623	103.89867	1.039878e

	count	unique	top	freq
month	226903	111	2024-07-01	3036
town	226903	26	SENGKANG	18497
flat_type	226903	7	4 ROOM	96251
block	226903	2752	2	685
street_name	226903	576	YISHUN RING RD	3233
storey_range	226903	17	04 TO 06	52041
flat_model	226903	21	Model A	81268
remaining_lease	226903	697	94 years 10 months	1919
address	226903	9690	308A PUNGGOL WALK	174

In [5]:
```
target_summary = pd.Series({
    "rows": len(df),
    "min_price": df["resale_price"].min(),
    "median_price": df["resale_price"].median(),
    "mean_price": df["resale_price"].mean(),
    "max_price": df["resale_price"].max(),
    "std_price": df["resale_price"].std(),
})
display(target_summary)

plt.figure(figsize=(5, 4))
plt.hist(df["resale_price"], bins=60)
plt.title("Distribution of Resale Price")
plt.xlabel("Resale Price")
plt.ylabel("Count")
plt.show()
```

```
rows            2.269030e+05
min_price       1.400000e+05
median_price    4.958880e+05
mean_price      5.272772e+05
max_price       1.700000e+06
std_price       1.880035e+05
dtype: float64
```



2. Feature engineering

Raw administrative datasets rarely arrive in the exact form that a predictive model needs.
This section turns the transaction table into a richer modeling dataset.

Purpose of feature engineering here

The goal is to create variables that reflect the economics of HDB pricing more directly, including:

- **time of sale**: captures overall market movement
- **property age / lease balance**: approximates remaining economic value
- **storey level**: captures vertical premium effects
- **flat size**: one of the strongest direct drivers of price
- **location**: captures spatial differences in demand
- **price per square meter (`psm`)**: useful for exploratory benchmarking

Why feature engineering matters so much

Machine-learning models do not understand housing logic on their own.
They only see the variables we choose to provide.

For property data, thoughtful feature engineering often matters just as much as model choice because it:

- converts text fields into learnable quantities
- makes hidden structure explicit
- gives simpler models a better chance to perform well
- improves interpretability later when explaining model outputs

Function rationale: translating raw text fields into numeric features

Several useful HDB variables are stored as text rather than as ready-to-model numbers.
Examples include:

- `remaining_lease`
- `storey_range`

Models generally work better when those columns are converted into numeric quantities.
So the next functions extract information such as:

- **remaining lease in months**
- **midpoint of the storey range**

This is a classic but important modeling step:
turning human-readable administrative labels into machine-readable signals without losing too much meaning.

In [6]:
```
def parse_remaining_lease_to_months(text):
    """Convert lease text into a single numeric value measured in months."""
    if pd.isna(text):
        return np.nan

    text = str(text).strip().lower()
    years = 0
    months = 0

    import re
    y = re.search(r"(\d+)\s+year", text)
    m = re.search(r"(\d+)\s+month", text)

    if y:
        years = int(y.group(1))
    if m:
        months = int(m.group(1))

    return years * 12 + months

def parse_storey_mid(text):
    """Approximate a storey-range string by its midpoint for numeric modeling."""
    if pd.isna(text):
        return np.nan
    try:
        low, _, high = str(text).partition(" TO ")
        return (int(low) + int(high)) / 2
    except Exception:
        return np.nan

model_df_full = df.copy()

# time features
model_df_full["month"] = pd.to_datetime(model_df_full["month"], format="%Y-%m-%d")
model_df_full["sale_year"] = model_df_full["month"].dt.year
model_df_full["sale_month_num"] = model_df_full["month"].dt.month
# model_df_full["sale_quarter"] = model_df_full["month"].dt.quarter

# lease / age features
model_df_full["remaining_lease_months"] = model_df_full["remaining_lease"].apply(parse_remaining_lease_to_months)
model_df_full["property_age_at_sale"] = model_df_full["sale_year"] - model_df_full["lease_commence_date"]
```

```python
# floor / area related
model_df_full["storey"] = model_df_full["storey_range"].apply(parse_storey_mid)

# simplified location key
model_df_full["address"] = model_df_full["block"].astype(str) + " " + model_df_full["street_name"].astype(str)

# log target for optional analysis if needed
model_df_full["log_resale_price"] = np.log1p(model_df_full["resale_price"])

model_df_full.head()
```

Out[6]:

	month	town	flat_type	block	street_name	storey_range	floor_area_sqm	flat_model	lease_commence_date	remaining_leas
0	2017-01-01	ANG MO KIO	4 ROOM	207	ANG MO KIO AVE 1	04 TO 06	97.0	New Generation	1976	58 years 0 month
1	2017-01-01	ANG MO KIO	3 ROOM	218	ANG MO KIO AVE 1	04 TO 06	67.0	New Generation	1976	58 years 0 month
2	2017-01-01	ANG MO KIO	3 ROOM	219	ANG MO KIO AVE 1	07 TO 09	67.0	New Generation	1977	59 years 0 month
3	2017-01-01	ANG MO KIO	4 ROOM	304	ANG MO KIO AVE 1	10 TO 12	97.0	New Generation	1977	59 years 0 month
4	2017-01-01	ANG MO KIO	5 ROOM	310A	ANG MO KIO AVE 1	10 TO 12	121.0	Improved	2012	94 years 0 month

5 rows × 22 columns

In [7]:

```python
engineered_numeric = [
    "floor_area_sqm",
    # "lease_commence_date",
    # "remaining_lease_months",
    "property_age_at_sale",
    "storey",
    "sale_year",
    "sale_month_num",
    # "sale_quarter",
    "latitude",
    "longitude",
]

engineered_categorical = [
    "town",
    "flat_type",
    "flat_model",
    "street_name",
    # "storey_range",
]

keep_cols = engineered_numeric + engineered_categorical + ["resale_price", "psm", "log_resale_price"]
model_df = model_df_full[keep_cols].copy()

print(model_df.shape)
model_df.head()
```

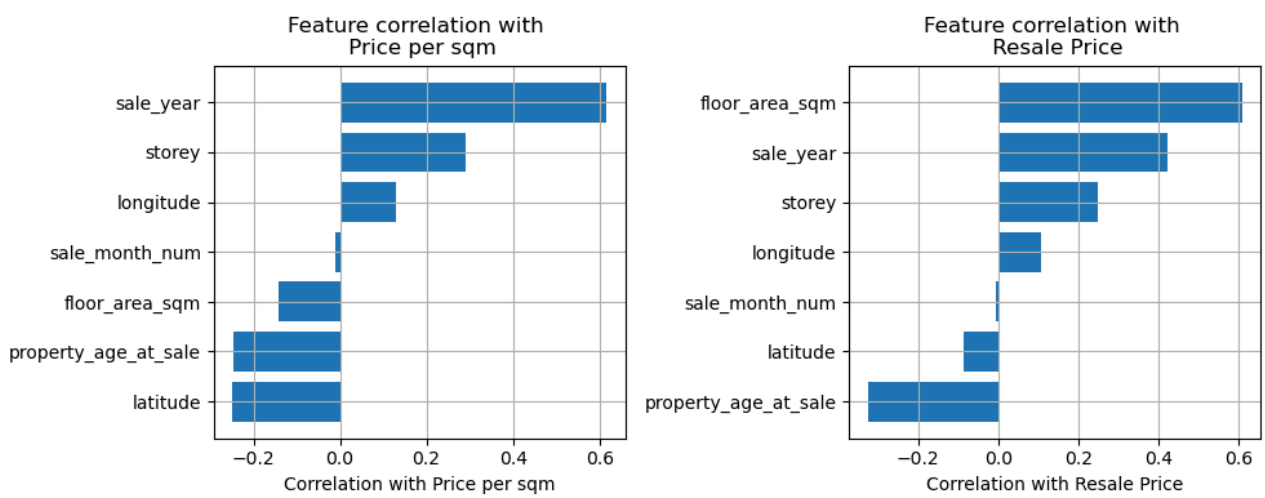
```
(226903, 14)
```

Out[7]:

	floor_area_sqm	property_age_at_sale	storey	sale_year	sale_month_num	latitude	longitude	town	flat_type	flat_model	st
0	97.0	41	5.0	2017	1	1.36582	103.84285	ANG MO KIO	4 ROOM	New Generation	
1	67.0	41	5.0	2017	1	1.36512	103.84174	ANG MO KIO	3 ROOM	New Generation	
2	67.0	40	8.0	2017	1	1.36598	103.84065	ANG MO KIO	3 ROOM	New Generation	
3	97.0	40	11.0	2017	1	1.36629	103.84499	ANG MO KIO	4 ROOM	New Generation	
4	121.0	5	11.0	2017	1	1.36481	103.84384	ANG MO KIO	5 ROOM	Improved	

In [8]:

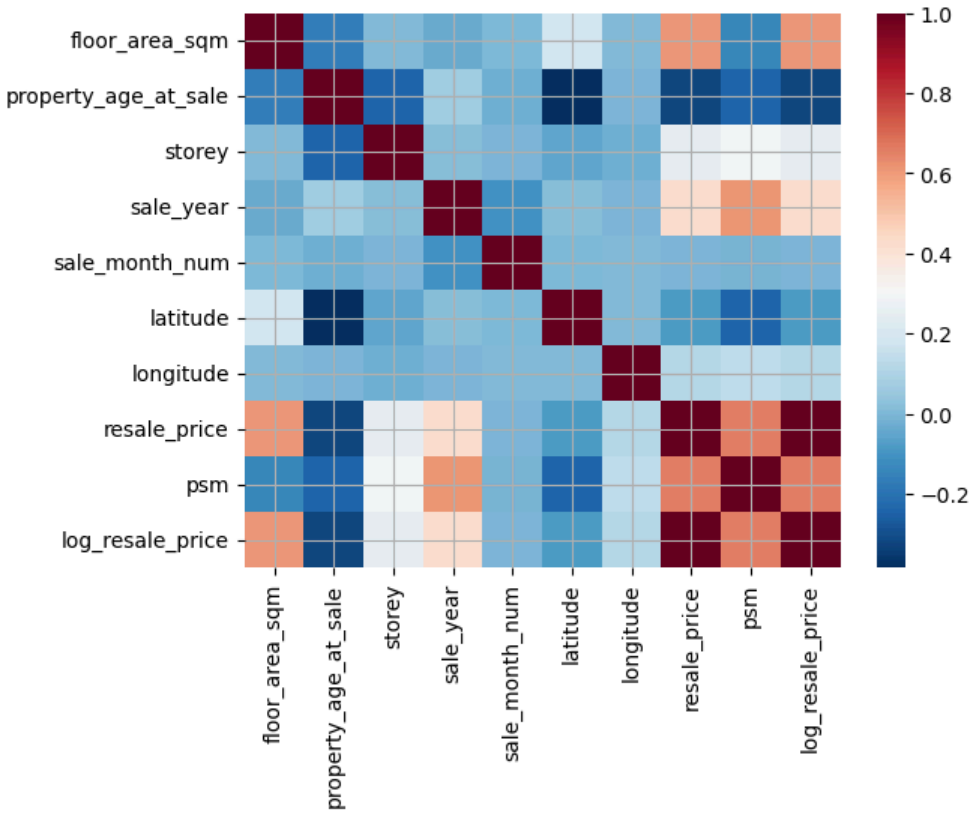
```
corr1 = model_df.select_dtypes(include=(['int32', 'int64', 'float64'])).corr('spearman')['psm'].drop(["psm", "resale_
corr2 = model_df.select_dtypes(include=(['int32', 'int64', 'float64'])).corr('spearman')['resale_price'].drop(["psm",

f, (ax1, ax2) =  plt.subplots(nrows=1, ncols=2, figsize=(10,4))

ax1.barh(corr1.index, corr1.values)
ax1.set_xlabel("Correlation with Price per sqm")
ax1.set_title("Feature correlation with \n Price per sqm")

ax2.barh(corr2.index, corr2.values)
ax2.set_xlabel("Correlation with Resale Price")
ax2.set_title("Feature correlation with \n Resale Price")

plt.tight_layout()
plt.show()
```



In [9]:

```
corr3 = model_df.select_dtypes(include=(['int32', 'int64', 'float64'])).corr('spearman')
sns.heatmap(corr3, cmap='RdBu_r')
```

Out[9]: <Axes: >



3. Train, Validate, Test Data

This section assigns different subsets of the data to different modeling roles.

Why use three splits instead of one?

- **training set**: used to fit model parameters
- **validation set**: used to compare hyperparameters and model families
- **test set**: used only for the final unbiased evaluation

That separation is critical.
If the test set is repeatedly used during tuning, model selection can gradually overfit to the evaluation data, giving overly optimistic results.

Practical interpretation

Because the purpose of this notebook is valuation-like prediction, the held-out test set is especially important.
It is the best approximation in the notebook to the real-world question:

> *How well would the model perform on transactions it has never seen before?*

In [10]:
```
X = model_df.drop(columns=["resale_price", "psm", "log_resale_price"])
y = model_df["resale_price"]

X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y, test_size=0.15, random_state=RANDOM_STATE
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full, test_size=0.1765, random_state=RANDOM_STATE
)
# This gives about 70% train / 15% val / 15% test overall.

print("Train:", X_train.shape, y_train.shape)
```

```
print("Validation:", X_val.shape, y_val.shape)
print("Test:", X_test.shape, y_test.shape)
```

```
Train: (158825, 11) (158825,)
Validation: (34042, 11) (34042,)
Test: (34036, 11) (34036,)
```

4. Helper functions and preprocessors

The next section defines reusable utilities for metrics, tuning, fitting, and preprocessing.

Why centralize this logic?

Without helper functions, every model section would need to repeat the same boilerplate for:

- fitting
- prediction
- metric calculation
- timing
- validation tuning
- result storage

Centralizing those steps makes the notebook:

- easier to read
- easier to debug
- easier to extend with additional models later
- more consistent across experiments

That consistency matters because fair model comparison requires the same evaluation workflow across all candidate methods.

Function rationale: common metrics and reusable evaluation helpers

The helper functions below create the evaluation backbone of the notebook.

Why use multiple regression metrics?

No single number tells the whole story:

- **RMSE** penalizes large misses more strongly
- **MAE** captures the average absolute error in dollar terms
- **R^2** measures how much variation the model explains
- **MAPE** expresses error as a percentage, which is often intuitive for pricing tasks

This combination is useful because a model intended for resale-price estimation should be judged from several angles, not only from one score.

Why store fitted models and predictions?

Later sections need to:

- compare models side by side
- draw actual-vs-predicted plots
- interpret the strongest tree-based model
- reuse the winning model for a real flat example

Keeping those objects in shared dictionaries makes the later notebook sections much more transparent and reusable.

In [11]:
```
results = []
fitted_models = {}
predictions = {}
```

```python
def regression_metrics(y_true, y_pred):
    """Compute a small set of complementary regression metrics."""
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    nonzero_mask = y_true != 0
    mape = np.mean(np.abs((y_true[nonzero_mask] - y_pred[nonzero_mask]) / y_true[nonzero_mask])) * 100

    return {
        "RMSE": rmse,
        "MAE": mae,
        "R2": r2,
        "MAPE_pct": mape,
    }

def add_result(model_name, fitted_model, X_eval, y_eval, fit_seconds=None, notes=None):
    """Evaluate a fitted model and store both metrics and predictions for later use."""
    pred = fitted_model.predict(X_eval)
    metrics = regression_metrics(y_eval, pred)
    metrics["Model"] = model_name
    if fit_seconds is not None:
        metrics["Fit_seconds"] = fit_seconds
    if notes is not None:
        metrics["Notes"] = notes
    results.append(metrics)
    fitted_models[model_name] = fitted_model
    predictions[model_name] = pred
    return pred, metrics

def sample_rows(X_df, y_series, max_rows, random_state=RANDOM_STATE):
    """Randomly subsample large datasets to keep expensive models practical."""
    if (max_rows is None) or (len(X_df) <= max_rows):
        return X_df.copy(), y_series.copy()
    sampled_index = X_df.sample(n=max_rows, random_state=random_state).index
    return X_df.loc[sampled_index].copy(), y_series.loc[sampled_index].copy()

def evaluate_on_validation(pipeline, X_tr, y_tr, X_va, y_va):
    """Fit one pipeline on the training split and score it on the validation split."""
    pipeline.fit(X_tr, y_tr)
    pred = pipeline.predict(X_va)
    return regression_metrics(y_va, pred)

def tune_with_progress(
    model_name,
    pipeline_builder,
    param_distributions,
    X_tr,
    y_tr,
    X_va,
    y_va,
    n_iter=8,
    sample_limit=None,
    random_state=RANDOM_STATE
):
    """Sample hyperparameters, evaluate them on the validation set, and keep the best one."""
    X_tr_use, y_tr_use = X_tr, y_tr
    if sample_limit is not None:
        X_tr_use, y_tr_use = sample_rows(X_tr, y_tr, sample_limit, random_state=random_state)

    sampled_params = list(ParameterSampler(param_distributions, n_iter=n_iter, random_state=random_state))

    tuning_rows = []
    best_score = np.inf
    best_params = None
    best_pipeline = None

    print(f"Tuning {model_name} on {len(X_tr_use):,} training rows and {len(X_va):,} validation rows")

    for params in tqdm(sampled_params, desc=f"{model_name} tuning"):
        pipeline = pipeline_builder(**params)
        start = time.time()
```

```
        metrics = evaluate_on_validation(pipeline, X_tr_use, y_tr_use, X_va, y_va)
        elapsed = time.time() - start

        row = {"fit_seconds": elapsed, **params, **metrics}
        tuning_rows.append(row)

        if metrics["RMSE"] < best_score:
            best_score = metrics["RMSE"]
            best_params = params
            best_pipeline = pipeline

    tuning_df = pd.DataFrame(tuning_rows).sort_values("RMSE").reset_index(drop=True)
    return best_params, tuning_df, best_pipeline

def fit_model_on_sample(pipeline, X_train_data, y_train_data, sample_limit=None, desc=None):
    """Fit a pipeline on all rows or on a controlled sample and return runtime metadata."""
    X_fit, y_fit = sample_rows(X_train_data, y_train_data, sample_limit, random_state=RANDOM_STATE)
    if desc is not None:
        print(f"{desc}: fitting on {len(X_fit):,} rows")
    start = time.time()
    pipeline.fit(X_fit, y_fit)
    fit_seconds = time.time() - start
    return pipeline, fit_seconds, len(X_fit)
```

Preprocessing rationale: why there are two preprocessors

The notebook defines two preprocessing pipelines because different model families have different numerical needs.

- **general preprocessor**
  imputes missing values, scales numeric inputs, and one-hot encodes categoricals
  → useful for linear models, k-NN, and neural networks

- **tree preprocessor**
  imputes missing values and one-hot encodes categoricals, but does **not** scale numerics
  → appropriate for Random Forest, XGBoost, and LightGBM

Why this matters

Scaling is essential for distance-based and gradient-based models, but it is usually unnecessary for tree models.
Using separate preprocessors keeps each model family in a setting that suits its learning mechanics.

This is a subtle but important sign of good ML workflow:
the notebook is not forcing every algorithm through the exact same data treatment when that would be suboptimal.

In [12]:
```
# General preprocessing
numeric_features = engineered_numeric
categorical_features = engineered_categorical

general_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler())
        ]), numeric_features),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", min_frequency=20))
        ]), categorical_features),
    ]
)

# Tree-friendly preprocessing: impute numerics, one-hot categoricals, no numeric scaling needed
tree_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median"))
        ]), numeric_features),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
            ("onehot", OneHotEncoder(handle_unknown="ignore", min_frequency=20))
        ]), categorical_features),
    ]
)
```

5. Linear Regression

Linear regression is the baseline model.

Why start here?

It is simple, fast, and interpretable.
It answers the question:

> *How far can we get if resale price is modeled as an approximately linear combination of the available features?*

That makes it a useful benchmark.
More complex models should earn their complexity by meaningfully improving on this baseline.

In [13]:
```
linear_model = Pipeline(steps=[
    ("preprocessor", general_preprocessor),
    ("model", LinearRegression())
])

start = time.time()
linear_model.fit(X_train_full, y_train_full)
linear_fit_seconds = time.time() - start

linear_pred, linear_metrics = add_result(
    "Linear Regression",
    linear_model,
    X_test,
    y_test,
    fit_seconds=linear_fit_seconds
)

linear_metrics
```

Out[13]:
```
{'RMSE': np.float64(55038.47410359727),
 'MAE': 41589.15133060245,
 'R2': 0.9157567331115394,
 'MAPE_pct': np.float64(8.684028714952328),
 'Model': 'Linear Regression',
 'Fit_seconds': 5.552926778793335}
```

6. Polynomial Regression

Polynomial regression extends the linear baseline by allowing selected numeric relationships to bend rather than remain perfectly straight.

Why try this?

HDB prices are unlikely to be strictly linear in variables such as:

- floor area
- age
- timing
- location-related effects

Polynomial features let the model capture some curvature while still staying within a fairly interpretable regression framework.

What this comparison tells us

If polynomial regression improves substantially over plain linear regression, that is early evidence that the pricing surface is meaningfully non-linear even before moving to tree ensembles.

In [14]:
```
poly_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("poly", PolynomialFeatures(degree=2, include_bias=False)),
            ("scaler", StandardScaler())
        ]), numeric_features),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", min_frequency=20))
        ]), categorical_features),
    ]
)

poly_model = Pipeline(steps=[
    ("preprocessor", poly_preprocessor),
    ("model", LinearRegression())
])

start = time.time()
poly_model.fit(X_train_full, y_train_full)
poly_fit_seconds = time.time() - start

poly_pred, poly_metrics = add_result(
    "Polynomial Regression",
    poly_model,
    X_test,
    y_test,
    fit_seconds=poly_fit_seconds
)

poly_metrics
```

Out[14]:
```
{'RMSE': np.float64(43530.76452675881),
 'MAE': 32563.047889193065,
 'R2': 0.9473018924314818,
 'MAPE_pct': np.float64(6.5069219442904185),
 'Model': 'Polynomial Regression',
 'Fit_seconds': 22.695796012878418}
```

7. Random Forest

Random Forest averages predictions from many decision trees.

Why it is a sensible candidate here

Housing prices often involve:

- non-linear relationships
- threshold effects
- interactions between size, age, town, and location

Random Forest can capture those patterns without requiring heavy manual feature transformation.
It is also robust on mixed tabular data and provides feature-importance outputs that are useful later for interpretation.

Model-building rationale: Random Forest

The next function builds a complete **pipeline**, not just a standalone model object.

That is important because preprocessing and prediction then stay bundled together:

- training data are transformed consistently
- validation and test data receive the exact same treatment
- inference later uses the same preprocessing rules as training

- tuning becomes much safer and easier to reproduce

For a notebook that aims to support practical price estimation, this end-to-end pipeline structure is much better than manually transforming data in separate steps.

In [15]:
```
# Construct a full preprocessing-plus-model pipeline for random forest regression.
def build_rf_pipeline(
    n_estimators=100,
    max_depth=20,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features="sqrt",
):
    return Pipeline(steps=[
        ("preprocessor", tree_preprocessor),
        ("model", RandomForestRegressor(
            n_estimators=n_estimators,
            max_depth=max_depth,
            min_samples_split=min_samples_split,
            min_samples_leaf=min_samples_leaf,
            max_features=max_features,
            bootstrap=True,
            random_state=RANDOM_STATE,
            n_jobs=-1
        ))
    ])

rf_param_distributions = {
    "n_estimators": [60, 100, 140],
    "max_depth": [14, 20, 28],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", 0.5],
}

rf_best_params, rf_tuning_df, _ = tune_with_progress(
    model_name="Random Forest",
    pipeline_builder=build_rf_pipeline,
    param_distributions=rf_param_distributions,
    X_tr=X_train,
    y_tr=y_train,
    X_va=X_val,
    y_va=y_val,
    n_iter=6,
    sample_limit=30000
)

display(rf_tuning_df.head(10))

print("Best RF params:", rf_best_params)

best_rf_model = build_rf_pipeline(**rf_best_params)

RF_FINAL_SAMPLE = 60000
best_rf_model, rf_fit_seconds, rf_rows_used = fit_model_on_sample(
    best_rf_model,
    X_train_full,
    y_train_full,
    sample_limit=RF_FINAL_SAMPLE,
    desc="Random Forest"
)

rf_pred, rf_metrics = add_result(
    "Random Forest",
    best_rf_model,
    X_test,
    y_test,
    fit_seconds=rf_fit_seconds,
    notes=f"trained on {rf_rows_used:,} sampled rows"
)

rf_metrics
```

```
Tuning Random Forest on 30,000 training rows and 34,042 validation rows
```

```
Random Forest tuning:   0%|          | 0/6 [00:00<?, ?it/s]
```

	fit_seconds	n_estimators	min_samples_split	min_samples_leaf	max_features	max_depth	RMSE	MAE	R2
0	14.521551	100	5	2	0.5	20	38503.552105	26654.490087	0.957614
1	17.426852	140	5	4	0.5	28	40341.237177	27610.097602	0.953472
2	17.265572	140	2	1	0.5	14	42684.244071	30603.415534	0.947910
3	11.568375	100	2	1	sqrt	28	52297.568684	38525.811692	0.921805
4	2.775109	140	5	4	sqrt	28	66340.854111	48357.830286	0.874171
5	5.216121	100	2	1	sqrt	20	70996.022156	53564.292483	0.855892

```
Best RF params: {'n_estimators': 100, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.5, 'max_dept
h': 20}
Random Forest: fitting on 60,000 rows
```

Out[15]:
```
{'RMSE': np.float64(35118.712216210835),
 'MAE': 24328.015480525068,
 'R2': 0.9657011471804318,
 'MAPE_pct': np.float64(4.650745803051609),
 'Model': 'Random Forest',
 'Fit_seconds': 40.68718433380127,
 'Notes': 'trained on 60,000 sampled rows'}
```

8. k-Nearest Neighbors

k-NN predicts a price by looking at similar historical observations in feature space.

Why include it?

Conceptually, it resembles a basic valuation instinct:

> *What did similar flats sell for?*

That makes it a nice benchmark for resale pricing.
If similarity is defined well enough, k-NN can perform surprisingly well on structured tabular data.

Limitation to keep in mind

k-NN is very sensitive to:

- feature scaling
- irrelevant dimensions
- the definition of "nearby" in a high-dimensional space

So its performance is informative not only about the model itself, but also about how well the engineered features capture practical comparability between flats.

In [16]:
```python
# Construct a preprocessing-plus-model pipeline for k-nearest-neighbors regression.
def build_knn_pipeline(
    n_neighbors=20,
    weights="distance",
    p=2,
):
    return Pipeline(steps=[
        ("preprocessor", general_preprocessor),
        ("model", KNeighborsRegressor(
            n_neighbors=n_neighbors,
            weights=weights,
            p=p,
            n_jobs=-1
        ))
    ])

knn_param_distributions = {
```

```
    "n_neighbors": [5, 10, 20, 30, 40],
    "weights": ["uniform", "distance"],
    "p": [1, 2],
}

knn_best_params, knn_tuning_df, _ = tune_with_progress(
    model_name="KNN",
    pipeline_builder=build_knn_pipeline,
    param_distributions=knn_param_distributions,
    X_tr=X_train,
    y_tr=y_train,
    X_va=X_val,
    y_va=y_val,
    n_iter=8,
    sample_limit=40000
)

display(knn_tuning_df.head(10))

best_knn_model = build_knn_pipeline(**knn_best_params)

start = time.time()
best_knn_model.fit(X_train_full, y_train_full)
knn_fit_seconds = time.time() - start

knn_pred, knn_metrics = add_result(
    "k-Nearest Neighbors",
    best_knn_model,
    X_test,
    y_test,
    fit_seconds=knn_fit_seconds
)

knn_metrics
```

```
Tuning KNN on 40,000 training rows and 34,042 validation rows
KNN tuning:   0%|          | 0/8 [00:00<?, ?it/s]
```

	fit_seconds	weights	p	n_neighbors	RMSE	MAE	R2	MAPE_pct
0	22.670330	distance	1	5	46943.205330	33362.185320	0.936997	6.432984
1	22.756363	distance	2	5	47249.093814	33697.507974	0.936173	6.491305
2	23.254158	distance	1	10	47841.478182	34238.072534	0.934562	6.600653
3	24.120313	uniform	1	5	49409.842426	35339.414601	0.930202	6.806795
4	23.197914	distance	2	20	50263.652390	35548.408701	0.927768	6.818641
5	24.739036	distance	2	30	52673.695483	37080.015860	0.920676	7.098584
6	22.951252	distance	1	40	55629.168905	39901.880102	0.911524	7.717027
7	23.007147	uniform	1	20	55774.734202	40391.617216	0.911061	7.796469

Out[16]:
```
{'RMSE': np.float64(35337.29298337566),
 'MAE': 25091.06370038833,
 'R2': 0.9652728626465681,
 'MAPE_pct': np.float64(4.857363249098731),
 'Model': 'k-Nearest Neighbors',
 'Fit_seconds': 1.3028895854949951}
```

9. Neural Network (MLPRegressor)

This section tests a multilayer perceptron, a simple feed-forward neural network for regression.

Why test a neural network on tabular housing data?

A neural network offers another flexible non-linear function approximator that is very different from tree ensembles. It may detect patterns that simpler linear models miss.

At the same time, tabular datasets do not always favor neural nets over boosted trees.
So this comparison is useful in a practical sense:

- if the neural net wins, that suggests strong smooth non-linear structure
- if it does not, that reinforces how strong tree boosting often is on structured property data

In [17]:
```
# Construct a preprocessing-plus-model pipeline for a feed-forward neural network regressor.
def build_mlp_pipeline(
    hidden_layer_sizes=(64, 32),
    alpha=0.001,
    learning_rate_init=0.003,
):
    return Pipeline(steps=[
        ("preprocessor", general_preprocessor),
        ("model", MLPRegressor(
            hidden_layer_sizes=hidden_layer_sizes,
            alpha=alpha,
            learning_rate_init=learning_rate_init,
            max_iter=120,
            early_stopping=True,
            validation_fraction=0.1,
            n_iter_no_change=8,
            random_state=RANDOM_STATE
        ))
    ])

mlp_param_distributions = {
    "hidden_layer_sizes": [(32,), (64,), (64, 32)],
    "alpha": [1e-4, 1e-3, 1e-2],
    "learning_rate_init": [0.001, 0.003]
}

mlp_best_params, mlp_tuning_df, _ = tune_with_progress(
    model_name="MLP",
    pipeline_builder=build_mlp_pipeline,
    param_distributions=mlp_param_distributions,
    X_tr=X_train,
    y_tr=y_train,
    X_va=X_val,
    y_va=y_val,
    n_iter=5,
    sample_limit=25000
)

display(mlp_tuning_df.head(10))

print("Best MLP params:", mlp_best_params)

best_mlp_model = build_mlp_pipeline(**mlp_best_params)

MLP_FINAL_SAMPLE = 40000
best_mlp_model, mlp_fit_seconds, mlp_rows_used = fit_model_on_sample(
    best_mlp_model,
    X_train_full,
    y_train_full,
    sample_limit=MLP_FINAL_SAMPLE,
    desc="MLP"
)

mlp_pred, mlp_metrics = add_result(
    "Neural Network (MLP)",
    best_mlp_model,
    X_test,
    y_test,
    fit_seconds=mlp_fit_seconds,
    notes=f"trained on {mlp_rows_used:,} sampled rows"
)

mlp_metrics
```

```
Tuning MLP on 25,000 training rows and 34,042 validation rows
MLP tuning:   0%|          | 0/5 [00:00<?, ?it/s]
```

	fit_seconds	learning_rate_init	hidden_layer_sizes	alpha	RMSE	MAE	R2	MAPE_pct
0	30.286330	0.003	(64, 32)	0.0001	48095.256703	35835.977247	0.933866	7.263878
1	14.231804	0.003	(64,)	0.0001	178078.479014	148605.986105	0.093346	27.855873
2	10.007171	0.003	(32,)	0.0001	385202.541227	359679.412509	-3.242257	68.363089
3	13.669992	0.001	(64,)	0.0010	508573.224352	478766.220439	-6.394787	90.962953
4	10.466524	0.001	(32,)	0.0001	541064.194531	509713.466164	-7.369823	96.809477

```
Best MLP params: {'learning_rate_init': 0.003, 'hidden_layer_sizes': (64, 32), 'alpha': 0.0001}
MLP: fitting on 40,000 rows
```

Out[17]:
```
{'RMSE': np.float64(41945.560805442234),
 'MAE': 30339.875046485897,
 'R2': 0.9510700873257398,
 'MAPE_pct': np.float64(5.996018127465096),
 'Model': 'Neural Network (MLP)',
 'Fit_seconds': 52.612067461013794,
 'Notes': 'trained on 40,000 sampled rows'}
```

10. XGBoost

XGBoost is a gradient-boosted tree model and is often one of the strongest baseline choices for tabular prediction tasks.

Why it is promising here

Boosting builds trees sequentially so that each new tree focuses on residual errors left by earlier trees.
That often produces very strong performance on datasets like housing transactions, where:

- effects are non-linear
- feature interactions matter
- local pattern corrections can improve price accuracy

Model-building rationale: boosted trees

The next two sections test **XGBoost** and **LightGBM**, which are often among the most competitive methods for structured tabular data.

The tuning grids are intentionally **moderate rather than huge**.
That reflects a practical trade-off:

- enough variation to compare sensible parameter settings
- not so much that the notebook becomes unnecessarily slow or difficult to reuse

This is especially appropriate for an applied notebook whose goal is insight and practical prediction, not exhaustive benchmark optimization.

In [18]:
```
# Construct a preprocessing-plus-model pipeline for XGBoost regression.
def build_xgb_pipeline(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
):
    return Pipeline(steps=[
        ("preprocessor", tree_preprocessor),
        ("model", XGBRegressor(
            n_estimators=n_estimators,
            max_depth=max_depth,
            learning_rate=learning_rate,
            subsample=subsample,
            colsample_bytree=colsample_bytree,
            objective="reg:squarederror",
            random_state=RANDOM_STATE,
            n_jobs=-1
```

```
        ))
    ])

xgb_param_distributions = {
    "n_estimators": [200, 300, 500],
    "max_depth": [4, 6, 8],
    "learning_rate": [0.03, 0.05, 0.1],
    "subsample": [0.7, 0.8, 1.0],
    "colsample_bytree": [0.7, 0.8, 1.0],
}

xgb_best_params, xgb_tuning_df, _ = tune_with_progress(
    model_name="XGBoost",
    pipeline_builder=build_xgb_pipeline,
    param_distributions=xgb_param_distributions,
    X_tr=X_train,
    y_tr=y_train,
    X_va=X_val,
    y_va=y_val,
    n_iter=8,
    sample_limit=80000
)

display(xgb_tuning_df.head(10))

best_xgb_model = build_xgb_pipeline(**xgb_best_params)

start = time.time()
best_xgb_model.fit(X_train_full, y_train_full)
xgb_fit_seconds = time.time() - start

xgb_pred, xgb_metrics = add_result(
    "XGBoost",
    best_xgb_model,
    X_test,
    y_test,
    fit_seconds=xgb_fit_seconds
)

display(xgb_metrics)
```

```
Tuning XGBoost on 80,000 training rows and 34,042 validation rows
XGBoost tuning:   0%|          | 0/8 [00:00<?, ?it/s]
```

	fit_seconds	subsample	n_estimators	max_depth	learning_rate	colsample_bytree	RMSE	MAE	R2	MAP
0	3.597198	0.7	500	6	0.10	0.8	31229.655010	22718.318214	0.972116	4.43
1	3.196774	0.8	300	8	0.10	1.0	31239.400535	22553.539612	0.972099	4.39
2	4.611535	1.0	500	8	0.05	1.0	31779.304953	22971.148979	0.971126	4.47
3	3.643989	0.8	500	6	0.10	1.0	31789.295955	23128.210944	0.971108	4.51
4	2.779021	1.0	500	6	0.10	0.8	31977.021451	23219.316677	0.970766	4.51
5	5.507225	0.7	500	8	0.03	0.7	32610.374374	23768.107530	0.969596	4.63
6	2.638149	0.8	300	6	0.03	1.0	39958.431302	29197.967491	0.954351	5.67
7	3.030784	0.7	500	4	0.03	0.7	43634.298012	31860.096588	0.945565	6.15

```
{'RMSE': np.float64(31302.279783173006),
 'MAE': 22803.31119415839,
 'R2': 0.972750761630654,
 'MAPE_pct': np.float64(4.4560268214768755),
 'Model': 'XGBoost',
 'Fit_seconds': 7.107378244400024}
```

11. LightGBM

LightGBM is another gradient-boosted tree framework designed for efficiency and strong performance on large tabular datasets.

Why compare it with XGBoost?

Both are strong boosted-tree methods, but they differ in:

- tree-growth strategy
- speed / memory behavior
- hyperparameter sensitivity
- sometimes final accuracy

Testing both helps answer a genuinely empirical question:

> *For this HDB resale dataset, which boosting implementation learns the pricing structure more effectively?*

In [19]:

```
# Construct a preprocessing-plus-model pipeline for LightGBM regression.
def build_lgbm_pipeline(
    n_estimators=400,
    num_leaves=31,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
):
    return Pipeline(steps=[
        ("preprocessor", tree_preprocessor),
        ("model", LGBMRegressor(
            n_estimators=n_estimators,
            num_leaves=num_leaves,
            learning_rate=learning_rate,
            subsample=subsample,
            colsample_bytree=colsample_bytree,
            random_state=RANDOM_STATE,
            n_jobs=-1
        ))
    ])

lgbm_param_distributions = {
    "n_estimators": [250, 400, 600],
    "num_leaves": [31, 63, 127],
    "learning_rate": [0.03, 0.05, 0.1],
    "subsample": [0.7, 0.8, 1.0],
    "colsample_bytree": [0.7, 0.8, 1.0],
}

lgbm_best_params, lgbm_tuning_df, _ = tune_with_progress(
    model_name="LightGBM",
    pipeline_builder=build_lgbm_pipeline,
    param_distributions=lgbm_param_distributions,
    X_tr=X_train,
    y_tr=y_train,
    X_va=X_val,
    y_va=y_val,
    n_iter=8,
    sample_limit=80000
)

display(lgbm_tuning_df.head(10))

best_lgbm_model = build_lgbm_pipeline(**lgbm_best_params)

start = time.time()
best_lgbm_model.fit(X_train_full, y_train_full)
lgbm_fit_seconds = time.time() - start

lgbm_pred, lgbm_metrics = add_result(
    "LightGBM",
    best_lgbm_model,
    X_test,
    y_test,
    fit_seconds=lgbm_fit_seconds
)

display(lgbm_metrics)
```

```
Tuning LightGBM on 80,000 training rows and 34,042 validation rows
LightGBM tuning:   0%|          | 0/8 [00:00<?, ?it/s]
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.004452 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 1873
[LightGBM] [Info] Number of data points in the train set: 80000, number of used features: 561
[LightGBM] [Info] Start training from score 526781.099217
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.004098 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 1873
[LightGBM] [Info] Number of data points in the train set: 80000, number of used features: 561
[LightGBM] [Info] Start training from score 526781.099217
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000706 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1873
[LightGBM] [Info] Number of data points in the train set: 80000, number of used features: 561
[LightGBM] [Info] Start training from score 526781.099217
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000912 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1873
[LightGBM] [Info] Number of data points in the train set: 80000, number of used features: 561
[LightGBM] [Info] Start training from score 526781.099217
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000899 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1873
[LightGBM] [Info] Number of data points in the train set: 80000, number of used features: 561
[LightGBM] [Info] Start training from score 526781.099217
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000941 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1873
[LightGBM] [Info] Number of data points in the train set: 80000, number of used features: 561
[LightGBM] [Info] Start training from score 526781.099217
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000760 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1873
[LightGBM] [Info] Number of data points in the train set: 80000, number of used features: 561
[LightGBM] [Info] Start training from score 526781.099217
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.004034 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1873
[LightGBM] [Info] Number of data points in the train set: 80000, number of used features: 561
[LightGBM] [Info] Start training from score 526781.099217
```

	fit_seconds	subsample	num_leaves	n_estimators	learning_rate	colsample_bytree	RMSE	MAE	R2	MAP
0	4.815710	1.0	127	400	0.10	0.8	27449.228580	19530.130176	0.978458	3.8
1	4.768493	0.7	127	400	0.10	0.8	27449.228580	19530.130176	0.978458	3.8
2	4.701867	0.8	127	400	0.10	1.0	27653.192334	19706.412120	0.978137	3.8
3	6.998422	1.0	127	600	0.05	1.0	27733.090562	19737.499214	0.978011	3.8
4	7.592914	0.7	127	600	0.03	0.7	27797.032520	19869.482111	0.977909	3.8
5	4.333845	0.8	63	600	0.10	1.0	27806.105777	19917.042413	0.977895	3.8
6	3.670768	0.7	127	250	0.03	0.7	31700.099373	22863.054395	0.971270	4.4
7	3.786669	0.8	63	400	0.03	1.0	31759.784013	23035.956902	0.971161	4.4

```
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.007880 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 1989
[LightGBM] [Info] Number of data points in the train set: 192867, number of used features: 615
[LightGBM] [Info] Start training from score 527064.945201
```

```
{'RMSE': np.float64(26374.950536317425),
 'MAE': 18785.39133469995,
 'R2': 0.9806542425747931,
 'MAPE_pct': np.float64(3.670215051457331),
 'Model': 'LightGBM',
 'Fit_seconds': 6.7268078327178955}
```

12. Model comparison

Once all candidate models are trained, this section compares them on the held-out test set.

How to read the table

- **lower RMSE / MAE** → smaller dollar errors
- **higher R^2** → more variance explained
- **lower MAPE** → smaller average percentage error

Why this section matters

This is where model selection becomes evidence-based.
Instead of assuming that a more complex model is better, the notebook asks the data to show which method generalizes best.

In [20]:
```
results_df = pd.DataFrame(results)

if len(results_df) > 0:
    ordered_cols = ["Model", "RMSE", "MAE", "R2", "MAPE_pct", "Fit_seconds", "Notes"]
    existing_cols = [col for col in ordered_cols if col in results_df.columns]
    results_df = results_df[existing_cols].sort_values("RMSE").reset_index(drop=True)

display(results_df)

best_model_name = results_df.iloc[0]["Model"]
print("Best model based on lowest test RMSE:", best_model_name)
```

	Model	RMSE	MAE	R2	MAPE_pct	Fit_seconds	Notes
0	LightGBM	26374.950536	18785.391335	0.980654	3.670215	6.726808	NaN
1	XGBoost	31302.279783	22803.311194	0.972751	4.456027	7.107378	NaN
2	Random Forest	35118.712216	24328.015481	0.965701	4.650746	40.687184	trained on 60,000 sampled rows
3	k-Nearest Neighbors	35337.292983	25091.063700	0.965273	4.857363	1.302890	NaN
4	Neural Network (MLP)	41945.560805	30339.875046	0.951070	5.996018	52.612067	trained on 40,000 sampled rows
5	Polynomial Regression	43530.764527	32563.047889	0.947302	6.506922	22.695796	NaN
6	Linear Regression	55038.474104	41589.151331	0.915757	8.684029	5.552927	NaN

```
Best model based on lowest test RMSE: LightGBM
```

Interpreting the comparison results

The comparison table shows a very clear ranking:

- **LightGBM is the strongest overall model**
- **XGBoost is also strong, but not quite as accurate here**
- simpler baselines such as linear and polynomial regression still perform reasonably well
- the best tree-based models materially outperform the simpler models, which suggests that HDB resale pricing is meaningfully **non-linear**

Using the displayed test metrics:

- **LightGBM RMSE ≈ 26.4k SGD**
- **LightGBM MAE ≈ 18.8k SGD**
- **LightGBM R^2 ≈ 0.981**

- **LightGBM MAPE ≈ 3.67%**

That is a strong result for a broad resale-price model built from transactional, structural, and location features.

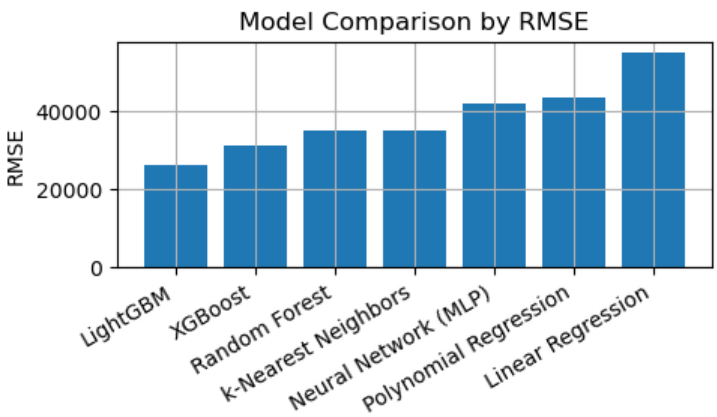
Practical takeaway

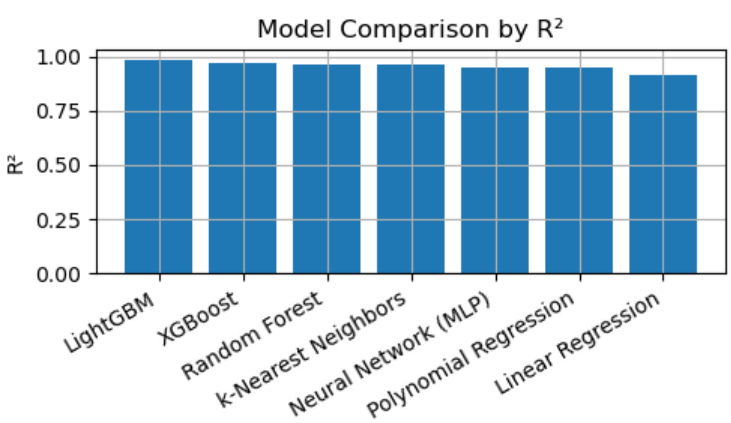
For valuation-style use, the most intuitive figures are often **MAE** and **MAPE**:

- an average absolute error of about **18.8k SGD**
- an average percentage error of about **3.7%**

This suggests the best model is accurate enough to be genuinely useful as a **pricing support tool**, while still not being a perfect substitute for a full market appraisal or current listing intelligence.

In [21]:
```
plt.figure(figsize=(5, 3))
plt.bar(results_df["Model"], results_df["RMSE"])
plt.xticks(rotation=30, ha="right")
plt.title("Model Comparison by RMSE")
plt.ylabel("RMSE")
plt.tight_layout()
plt.show()

plt.figure(figsize=(5, 3))
plt.bar(results_df["Model"], results_df["R2"])
plt.xticks(rotation=30, ha="right")
plt.title("Model Comparison by R²")
plt.ylabel("R²")
plt.tight_layout()
plt.show()
```





13. Actual vs predicted plots

Metric tables summarize performance, but scatter plots reveal *how* the errors are distributed.

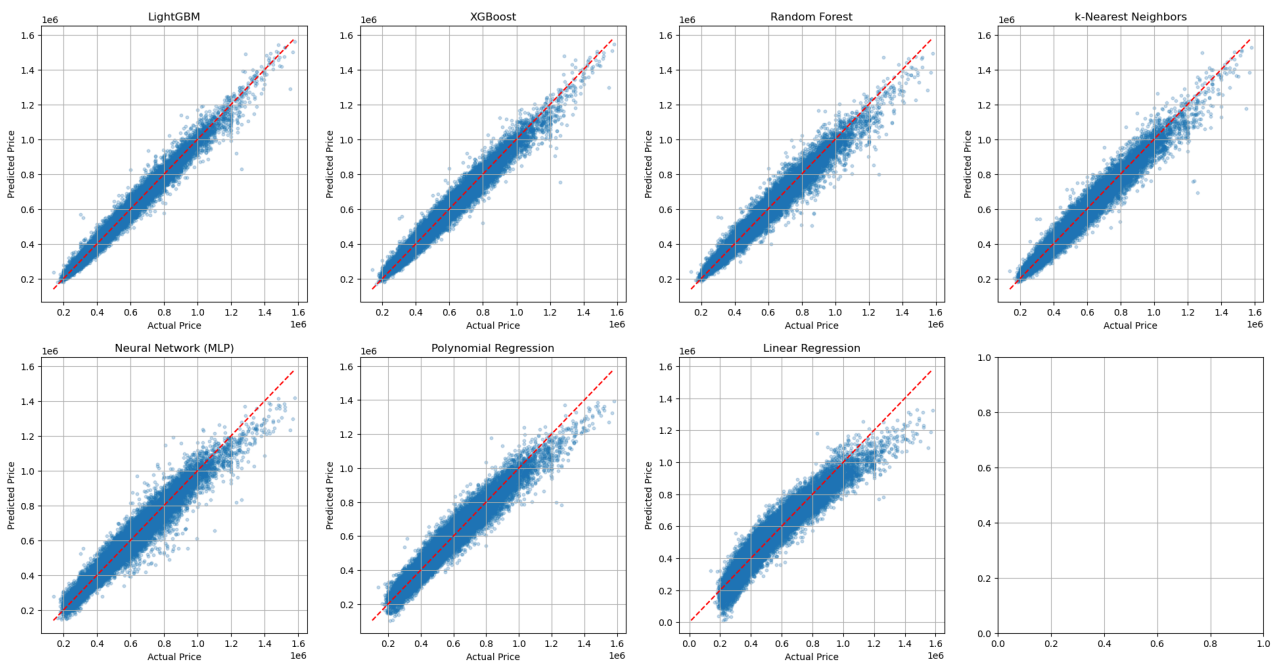
Why these plots matter

They show whether a model:

- follows the diagonal closely across the full price range
- systematically underpredicts expensive flats
- overpredicts cheaper flats
- becomes noisier in certain parts of the market

For a resale-price model, this matters because average accuracy alone can hide poor behavior at the higher-value end of the market.

In [22]:
```
f, ([ax1, ax2, ax3, ax4], [ax5, ax6, ax7, ax8]) =  plt.subplots(2, 4, figsize=(24,12))

for i in range(0,len(results_df["Model"])):
    model_name = results_df["Model"][i]
    axes =[ax1, ax2, ax3, ax4, ax5, ax6, ax7, ax8][i]

    pred = predictions[model_name]
    axes.scatter(y_test, pred, alpha=0.25, s=10)
    min_val = min(y_test.min(), pred.min())
    max_val = max(y_test.max(), pred.max())
    axes.plot([min_val, max_val], [min_val, max_val], linestyle="--", c="red")
    axes.set_title(f"{model_name}")
    axes.set_xlabel("Actual Price")
    axes.set_ylabel("Predicted Price")

plt.show()
```



How well can the model actually predict a real flat?

A very practical test is to feed the model the details of a real flat rather than just rely on aggregate test metrics.

Here the notebook checks a personal HDB transaction:

- **5-room flat**
- **Sengkang (Rivervale Crescent)**
- **purchased in August 2016**
- **actual transaction price: 415,000 SGD**

This is a meaningful sanity check because it asks whether the model gives a sensible answer for a real-world case that matters personally.

Important caveat

The notebook notes that this sale occurred before the main modeled period begins in January 2017, so this is slightly outside the training time range.
That makes the exercise more demanding and therefore a stronger real-world sanity check.

In [23]:
```
pred = best_lgbm_model.predict(pd.DataFrame([{
    'floor_area_sqm': 110,
    'property_age_at_sale': 15,
    'storey': 15,
    'sale_year': 2016,
    'sale_month_num': 8,
    'latitude': 1.38841338,
    'longitude': 103.9066762,
    'town': 'SENGKANG',
    'flat_type': '5 ROOM',
    'flat_model': 'Improved',
    'street_name': 'RIVERVALE CRES'
}]))

print(f'For my own HDB flat (5 Room, Sengkang, bought in Aug 2016):')
print(f"-- The predicted transaction price was: SGD {pred[0]:,.0f}")
print(f"-- My actual transaction price was: SGD 415,000")
```

```
For my own HDB flat (5 Room, Sengkang, bought in Aug 2016):
-- The predicted transaction price was: SGD 410,164
-- My actual transaction price was: SGD 415,000
```

The predicted value of **410,164 SGD** is very close to the actual purchase price of **415,000 SGD**.

Why this is encouraging

The exact match is not the point.
What matters is that the error is small relative to the transaction value, which suggests that the model has learned a market structure that is realistic enough to handle an individual flat example reasonably well.

That gives confidence to the next step: using the fitted model as a **current pricing guide** for the same flat under updated sale conditions.

In [24]:
```
pred = best_lgbm_model.predict(pd.DataFrame([{
    'floor_area_sqm': 110,
    'property_age_at_sale': 25,
    'storey': 15,
    'sale_year': 2026,
    'sale_month_num': 3,
    'latitude': 1.38841338,
    'longitude': 103.9066762,
    'town': 'SENGKANG',
    'flat_type': '5 ROOM',
    'flat_model': 'Improved',
    'street_name': 'RIVERVALE CRES'
}]))

print(f'For my own HDB flat, if I were to sell it today:')
print(f"-- The predicted transaction price will be: SGD {pred[0]:,.0f}")
```

```
For my own HDB flat, if I were to sell it today:
-- The predicted transaction price will be: SGD 660,170
```

14. Feature importance and SHAP

After identifying the strongest tree-based model, the notebook moves from **prediction** to **interpretation**.

Two complementary questions

1. **Which features matter most overall?**
   → answered by feature importance

2. **How do those features push specific predictions up or down?**
   → explored with SHAP

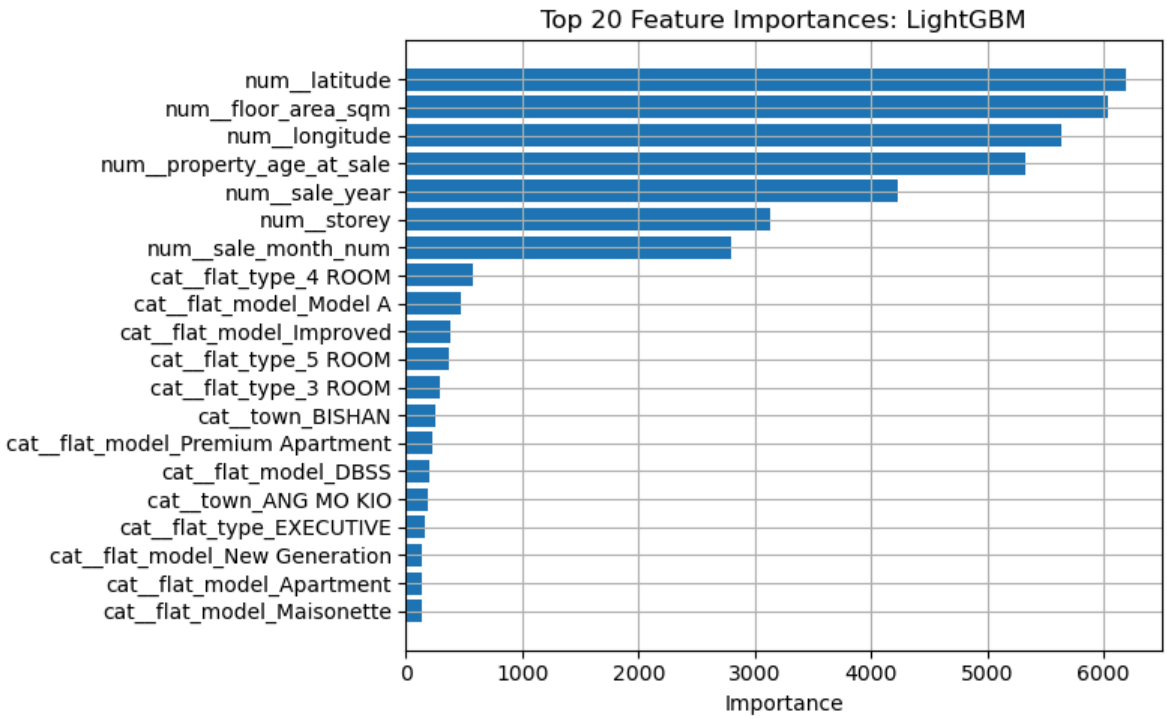
This is an important step because a high-performing model is more useful when we can also explain what it appears to be learning about HDB price formation.

In [25]:
```
tree_model_candidates = [name for name in results_df["Model"] if name in ["LightGBM", "XGBoost", "Random Forest"]]
best_tree_model_name = tree_model_candidates[0] if len(tree_model_candidates) > 0 else None
print("Best tree-based model:", best_tree_model_name)
```

```
Best tree-based model: LightGBM
```

In [26]:
```
best_tree_model = fitted_models[best_tree_model_name]
tree_pre = best_tree_model.named_steps["preprocessor"]
tree_est = best_tree_model.named_steps["model"]

feature_names = tree_pre.get_feature_names_out()

if hasattr(tree_est, "feature_importances_"):
    feature_importances = pd.DataFrame({
        "feature": feature_names,
        "importance": tree_est.feature_importances_
    }).sort_values("importance", ascending=False)

    display(feature_importances.head(20))

    top_n = 20
    top_feat = feature_importances.head(top_n).sort_values("importance")
    plt.figure(figsize=(8, 5))
    plt.barh(top_feat["feature"], top_feat["importance"])
    plt.title(f"Top {top_n} Feature Importances: {best_tree_model_name}")
    plt.xlabel("Importance")
    plt.tight_layout()
    plt.show()
```

	feature	importance
5	num__latitude	6191
0	num__floor_area_sqm	6036
6	num__longitude	5636
1	num__property_age_at_sale	5324
3	num__sale_year	4228
2	num__storey	3127
4	num__sale_month_num	2791
36	cat__flat_type_4 ROOM	576
48	cat__flat_model_Model A	475
45	cat__flat_model_Improved	375
37	cat__flat_type_5 ROOM	365
35	cat__flat_type_3 ROOM	286
9	cat__town_BISHAN	258
53	cat__flat_model_Premium Apartment	230
44	cat__flat_model_DBSS	205
7	cat__town_ANG MO KIO	182
38	cat__flat_type_EXECUTIVE	158
43	cat__flat_model_Apartment	141
52	cat__flat_model_New Generation	141
47	cat__flat_model_Maisonette	135



What the feature-importance results suggest

The top-ranked features are dominated by:

- **latitude**
- **floor area**
- **longitude**
- **property age at sale**
- **sale year**
- **storey**
- **sale month**

This is a very plausible result.

Interpretation

The model appears to be learning three major price drivers:

1. **Location matters enormously**
   Latitude and longitude ranking near the very top shows that price differences are not explained fully by town labels alone. Fine-grained spatial position contains major valuation signal.

2. **Size and age are core structural drivers**
   Larger flats tend to command higher prices, while older flats often face lease-related discounts.

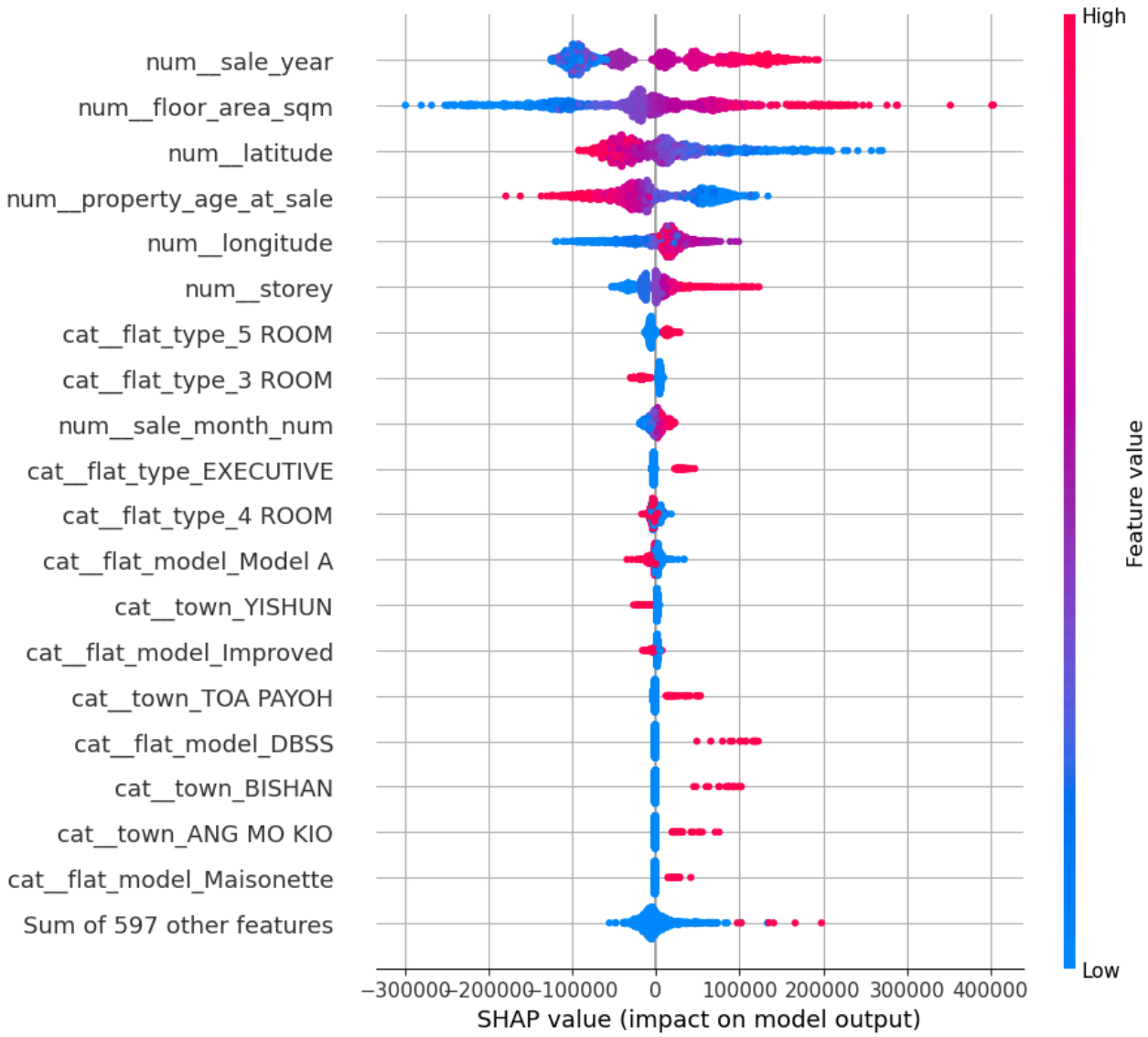
3. **Timing matters because the resale market moves over time**
   Sale year and sale month are useful because the HDB market is not static; prices shift with broader market conditions.

Overall, the feature ranking is economically intuitive, which strengthens confidence that the model is learning meaningful housing patterns rather than only statistical noise.

In [27]:
```
best_tree_model = fitted_models[best_tree_model_name]
tree_pre = best_tree_model.named_steps["preprocessor"]
tree_est = best_tree_model.named_steps["model"]

X_shap_sample, y_shap_sample = sample_rows(X_test, y_test, max_rows=1200, random_state=RANDOM_STATE)
X_shap_transformed = tree_pre.transform(X_shap_sample)
```

```python
if hasattr(X_shap_transformed, "toarray"):
    # Keep sample moderate to avoid excessive memory use
    X_shap_dense = X_shap_transformed.toarray()
else:
    X_shap_dense = X_shap_transformed

try:
    explainer = shap.Explainer(tree_est, X_shap_dense, feature_names=tree_pre.get_feature_names_out())
    shap_values = explainer(X_shap_dense)

    shap.plots.beeswarm(shap_values, max_display=20)
except Exception as e:
    print("SHAP calculation failed for this model/environment.")
    print("Details:", e)
```

```
100%|==================| 1194/1200 [00:39<00:00]
```



Why SHAP is especially useful here

Feature importance gives a global ranking, but SHAP adds direction and local detail.

For example, SHAP can help answer questions such as:

- does a larger floor area generally push the prediction upward?
- does older property age tend to pull value downward?
- how strongly does location change the estimate for a specific flat?

That makes SHAP particularly relevant for practical pricing use, because a seller usually wants not just a number, but also a rough explanation of **why** the model arrived there.

15. Dimension reduction for visualization

The feature space becomes fairly high-dimensional after preprocessing, especially once categorical variables are one-hot encoded.

Why include PCA, t-SNE, and UMAP?

These methods reduce that high-dimensional representation down to two dimensions so we can visually inspect broader structure such as:

- clustering of similar flats
- gradients associated with resale price
- separation between more common and more unusual transactions

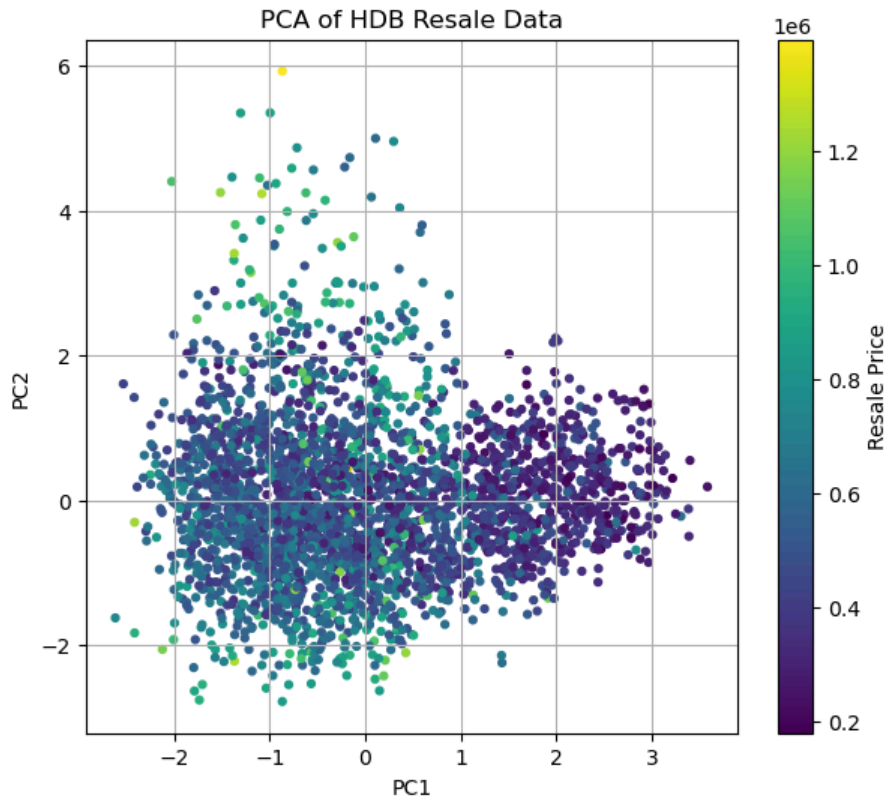
What this section is *not*

This is **not** part of the predictive pipeline used for the final price model.
It is mainly an exploratory / teaching section that helps build intuition about how structured the transformed feature space is.

In [28]:
```
sample_n = min(3000, len(model_df))
viz_df = model_df.sample(sample_n, random_state=RANDOM_STATE).copy()

X_viz = viz_df.drop(columns=["resale_price", "psm", "log_resale_price"])
y_viz = viz_df["resale_price"]

viz_preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler())
        ]), numeric_features),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", min_frequency=20))
        ]), categorical_features),
    ]
)

X_viz_transformed = viz_preprocessor.fit_transform(X_viz)

if hasattr(X_viz_transformed, "toarray"):
    X_viz_dense = X_viz_transformed.toarray()
else:
    X_viz_dense = X_viz_transformed

print("Visualization sample size:", len(viz_df))
print("Transformed visualization matrix shape:", X_viz_dense.shape)
```

```
Visualization sample size: 3000
Transformed visualization matrix shape: (3000, 70)
```

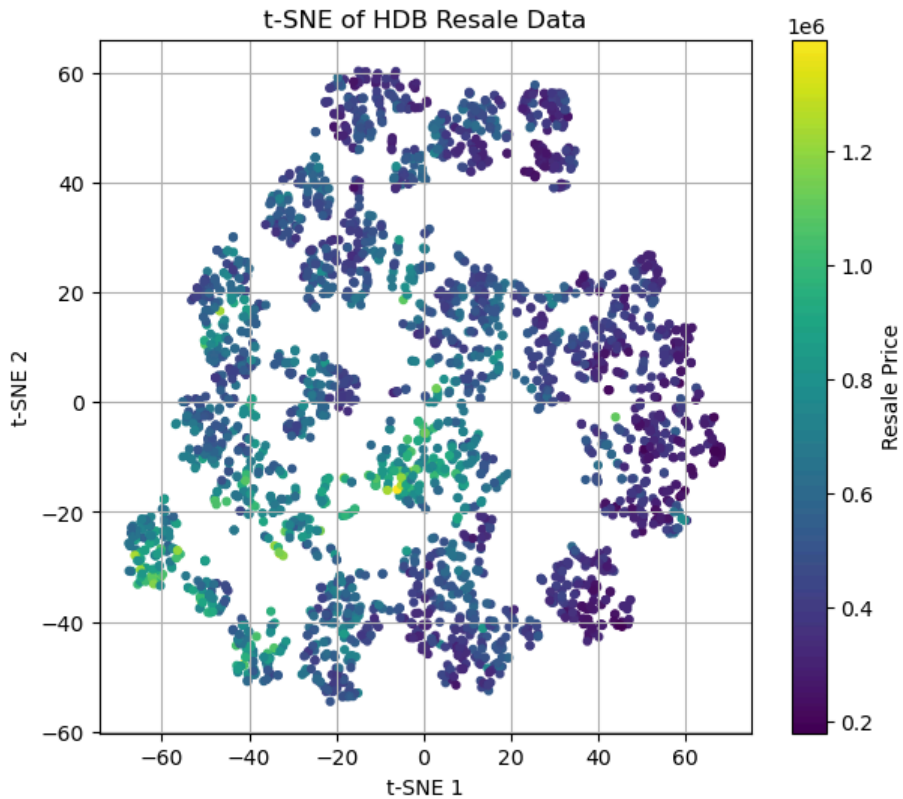
In [29]:
```
# PCA
pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_viz_dense)

plt.figure(figsize=(7, 6))
sc = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_viz, s=12)
plt.colorbar(sc, label="Resale Price")
plt.title("PCA of HDB Resale Data")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()

print("Explained variance ratio:", pca.explained_variance_ratio_)
```




```
Explained variance ratio: [0.16687614 0.12379585]
```

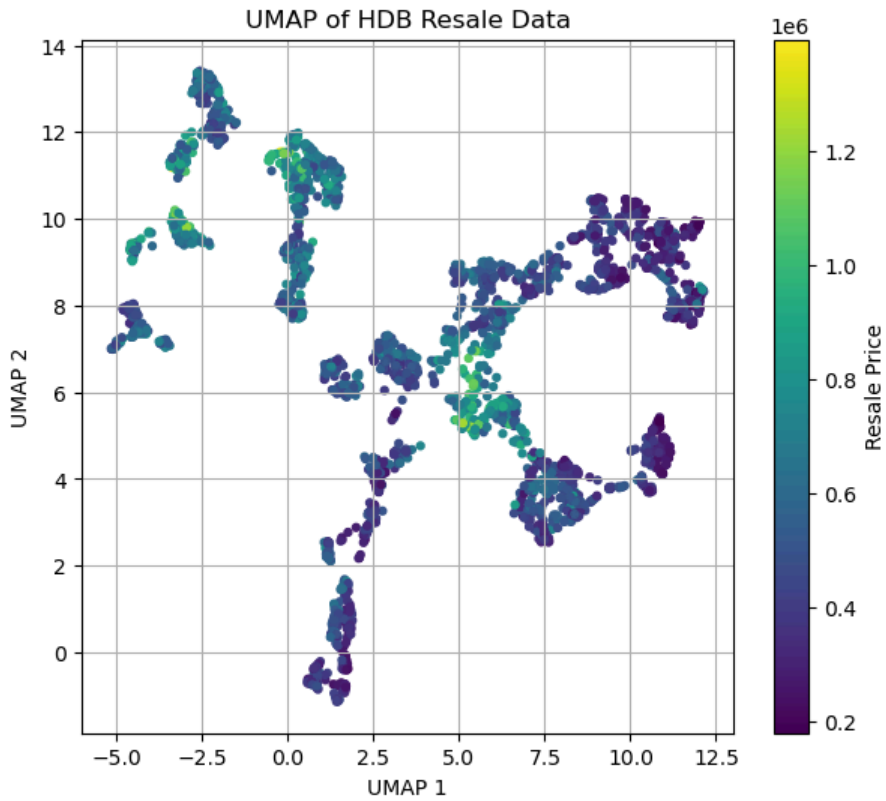
In [30]:

```python
# t-SNE
tsne = TSNE(
    n_components=2,
    perplexity=30,
    init="pca",
    learning_rate="auto",
    random_state=RANDOM_STATE
)
X_tsne = tsne.fit_transform(X_viz_dense)

plt.figure(figsize=(7, 6))
sc = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_viz, s=12)
plt.colorbar(sc, label="Resale Price")
plt.title("t-SNE of HDB Resale Data")
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.show()
```



In [31]:

```
# UMAP
reducer = umap.UMAP(n_components=2, random_state=RANDOM_STATE)
X_umap = reducer.fit_transform(X_viz_dense)

plt.figure(figsize=(7, 6))
sc = plt.scatter(X_umap[:, 0], X_umap[:, 1], c=y_viz, s=12)
plt.colorbar(sc, label="Resale Price")
plt.title("UMAP of HDB Resale Data")
plt.xlabel("UMAP 1")
plt.ylabel("UMAP 2")
plt.show()
```



Reading the low-dimensional maps carefully

If similar colors and nearby points tend to cluster together, that suggests the engineered features are capturing meaningful market structure.
In other words, flats with similar transformed profiles often do sit near one another in the learned representation.

At the same time, these plots should be interpreted cautiously:

- **PCA** preserves broad linear variation
- **t-SNE** emphasizes local neighborhoods
- **UMAP** often balances local and larger-scale structure

So the plots are useful for intuition, but they are not literal maps of Singapore and they should not be treated as causal evidence by themselves.

16. Final summary

This notebook shows that HDB resale pricing can be modeled quite effectively using a combination of structural, temporal, and geospatial features.

Main findings

- The dataset is large enough to support meaningful supervised learning.
- Simple baselines work reasonably well, which is a good sign for data quality.
- **Boosted tree models perform best**, with **LightGBM** emerging as the strongest model in this notebook.
- The winning model achieves errors that are small enough to make it genuinely useful as a practical pricing aid.
- The interpretation section shows that **location, size, age, timing, and storey** are all important contributors to price.

Answer to the core practical question

For the motivating use case — estimating a reasonable selling price for an HDB flat today — the notebook suggests that the answer is **yes**:

> a well-built model can provide a credible market-price estimate.

In the personal flat example, the model:

- reproduced the historical purchase price quite closely
- produced a current indicative selling estimate of roughly **660k SGD**

Final caution

This should still be treated as an **evidence-based estimate**, not as a guaranteed market value.
Actual achievable sale price can also depend on factors not fully captured here, such as:

- the exact renovation condition
- interior view / facing
- unusually recent nearby comparables
- changing short-term market sentiment
- negotiation dynamics

So the best use of this notebook is as a **data-driven valuation support tool**: a strong starting point for pricing, to be combined with current market judgment and comparable recent transactions.

In [32]:
```
display(results_df)

print("Best model:", best_model_name)
print()
print("Interpretation tips:")
print("- Lower RMSE / MAE is better.")
print("- Higher R² is better.")
print("- Random Forest and MLP in this notebook are intentionally trained on sampled rows to keep runtime practical.")
print("- If you want the absolute best accuracy later, you can increase RF_FINAL_SAMPLE and MLP_FINAL_SAMPLE gradually.")
print("- The geospatial maps use the geocoded latitude / longitude columns you added.")
print("- Tree boosting models often outperform plain linear models on housing data if the package is available.")
print("- KNN can also be slow when predicting on very large sets.")
```

	Model	RMSE	MAE	R2	MAPE_pct	Fit_seconds	Notes
0	LightGBM	26374.950536	18785.391335	0.980654	3.670215	6.726808	NaN
1	XGBoost	31302.279783	22803.311194	0.972751	4.456027	7.107378	NaN
2	Random Forest	35118.712216	24328.015481	0.965701	4.650746	40.687184	trained on 60,000 sampled rows
3	k-Nearest Neighbors	35337.292983	25091.063700	0.965273	4.857363	1.302890	NaN
4	Neural Network (MLP)	41945.560805	30339.875046	0.951070	5.996018	52.612067	trained on 40,000 sampled rows
5	Polynomial Regression	43530.764527	32563.047889	0.947302	6.506922	22.695796	NaN
6	Linear Regression	55038.474104	41589.151331	0.915757	8.684029	5.552927	NaN

```
Best model: LightGBM

Interpretation tips:
- Lower RMSE / MAE is better.
- Higher R² is better.
- Random Forest and MLP in this notebook are intentionally trained on sampled rows to keep runtime practical.
- If you want the absolute best accuracy later, you can increase RF_FINAL_SAMPLE and MLP_FINAL_SAMPLE gradually.
- The geospatial maps use the geocoded latitude / longitude columns you added.
- Tree boosting models often outperform plain linear models on housing data if the package is available.
- KNN can also be slow when predicting on very large sets.
```